

Ilmware Building Enterprise RAG Pipelines with Small, Specialized Models

Ilmware provides a unified framework for building LLM-based applications (e.g, RAG, Agents), using small, specialized models that can be deployed privately, integrated with enterprise knowledge sources safely and securely, and cost-effectively tuned and adapted for any business process. Ilmware has two main components: RAG Pipeline - integrated components for the full lifecycle of connecting knowledge sources to generative AI models; and 50+ small, specialized models fine-tuned for key tasks in enterprise process automation, including fact-based question-answering, classification, summarization, and extraction. By bringing together both of these components, along with integrating leading open source models and underlying technologies, Ilmware offers a comprehensive set of tools to rapidly build knowledge-based enterprise LLM applications. Most of our examples can be run without a GPU server - get started right away on your laptop. Join us on Discord | Watch Youtube Tutorials | Explore our Model Families on Huggingface

New to RAG? Check out the Fast Start video series

Multi-Model Agents with SLIM Models - Intro-Video

Intro to SLIM

Function Call Models Can't wait? Get SLIMs right away: from

```
Ilmware.models import ModelCatalog
ModelCatalog().get_ilm_toolkit() # get all SLIM models, delivered as small, fast quantized tools
ModelCatalog().tool_test_run("slim-sentiment-tool") # see the model in action with test script included
```

Key features

Writing code with Ilmware is based on a few main concepts:

- Model Catalog: Access all models the same way with easy lookup, regardless of underlying implementation.
- Library: ingest, organize and index a collection of knowledge at scale
- Parse, Text Chunk and Embed.
- Query: query libraries with mix of text, semantic, hybrid, metadata, and custom filters.
- Prompt with Sources: the easiest way to combine knowledge retrieval with a LLM inference.

RAG-Optimized Models

- 1-7B parameter models designed for RAG workflow integration and running locally.
- Simple-to-Scale Database Options - integrated data stores from laptop to parallelized cluster.
- Agents with Function Calls and SLIM Models

Start coding - Quick Start for RAG

What's New?

- Best New Small RAG Model - BLING finetune of Phi-3 - "bling-phi-3-gguf" - see the video
- Web Services with Agent Calls for Financial Research - end-to-end scenario - video and example
- Voice Transcription with WhisperCPP - getting_started, using_sample_files, and analysis_use_case with

great_speeches_video -Phi-3 GGUF Streaming Local Chatbot with UI -
setup your own Phi-3-gguf chatbot on your laptop in minutes - example with
video -Small, specialized, function-calling Extract Model - introducing slim-
extract - video and example -LLM to Answer Yes/No questions - introducing
slim-boolean model - video and example -Natural Language Query to CSV
End to End example - using slim-sql model - video and example and now
using Custom Tables on Postgres example -Multi-Model Agents with SLIM
models - multi-step Agents with SLIMs on CPU - video - example -OCR
Embedded Document Images Example - systematically extract text from
images embedded in documents example -Enhanced Parser Functions for
PDF, Word, Powerpoint and Excel - new text-chunking controls and
strategies, extract tables, images, header text - example -Agent Inference
Server - set up multi-model Agents over Inference Server example -GGUF -
Getting Started - check out examples - GGUF (example) and Videos video
-Optimizing Accuracy of RAG Prompts - check out example and videos -
part I and part II Getting Started Step 1 - Install llmware - pip3 install
llmware or pip3 install 'llmware[full]' note: starting with v0.3.0, we provide
options for a core install (minimal set of dependencies) or full install (adds to
the core with wider set of related python libraries). Step 2- Go to Examples -
Get Started Fast with 100+ 'Cut-and-Paste' Recipes Step 3 - Tutorial Videos
- check out our Youtube channel for high-impact 5-10 minute tutorials on the
latest examples. 🍌 Working with the llmware Github repository The llmware
repo can be pulled locally to get access to all the examples, or to work
directly with the latest version of the llmware code. git clone
git@github.com:llmware-ai/llmware.git We have provided a
welcome_to_llmware automation script in the root of the repository folder.
After cloning: On Windows command line: .
\\welcome_to_llmware_windows.sh On Mac / Linux command line: sh ./
welcome_to_llmware.sh Alternatively, if you prefer to complete setup
without the welcome automation script, then the next steps include: install
requirements.txt - inside the /llmware path - e.g., pip3 install -r llmware/
requirements.txt install requirements_extras.txt - inside the /llmware path -
e.g., pip3 install -r llmware/requirements_extras.txt (Depending upon your
use case, you may not need all or any of these installs, but some of these
will be used in the examples.) run examples - copy one or more of the
example .py files into the root project path. (We have seen several IDEs that
will attempt to run interactively from the nested /example path, and then not

have access to the /llmware module - the easy fix is to just copy the example you want to run into the root path). install vector db - no-install vector db options include milvus lite, chromadb, faiss and lancedb - which do not require a server install, but do require that you install the python sdk library for that vector db, e.g., pip3 install pymilvus, or pip3 install chromadb. If you look in examples/Embedding, you will see examples for getting started with various vector DB, and in the root of the repo, you will see easy-to-get-started docker compose scripts for installing milvus, postgres/pgvector, mongo, qdrant, neo4j, and redis. Note: we have seen recently issues with Pytorch==2.3 on some platforms - if you run into any issues, we have seen that uninstalling Pytorch and downleveling to Pytorch==2.1 usually solves the problem. Data Store Options Fast Start: use SQLite3 and ChromaDB (File-based) out-of-the-box - no install required Speed + Scale: use MongoDB (text collection) and Milvus (vector db) - install with Docker Compose Postgres: use Postgres for both text collection and vector DB - install with Docker Compose Mix-and-Match: LLMWare supports 3 text collection databases (Mongo, Postgres, SQLite) and 10 vector databases (Milvus, PGVector-Postgres, Neo4j, Redis, Mongo-Atlas, Qdrant, Faiss, LanceDB, ChromaDB and Pinecone) Meet our Models SLIM model series: small, specialized models fine-tuned for function calling and multi-step, multi-model Agent workflows. DRAGON model series: Production-grade RAG-optimized 6-7B parameter models - "Delivering RAG on ..." the leading foundation base models. BLING model series: Small CPU-based RAG-optimized, instruct-following 1B-3B parameter models. Industry BERT models: out-of-the-box custom trained sentence transformer embedding models fine-tuned for the following industries: Insurance, Contracts, Asset Management, SEC. GGUF Quantization: we provide 'gguf' and 'tool' versions of many SLIM, DRAGON and BLING models, optimized for CPU deployment. Using LLMs and setting-up API keys & secrets LLMWare is an open platform and supports a wide range of open source and proprietary models. To use LLMWare, you do not need to use any proprietary LLM - we would encourage you to experiment with SLIM, BLING, DRAGON, Industry-BERT, the GGUF examples, along with bringing in your favorite models from HuggingFace and Sentence Transformers. If you would like to use a proprietary model, you will need to provide your own API Keys. API keys and secrets for models, aws, and pinecone can be set-up for use in environment variables or passed directly to method calls. Roadmap - Where

are we going ... Interested in contributing to llmware? Information on ways to participate can be found in our Contributors Guide. As with all aspects of this project, contributing is governed by our Code of Conduct. Questions and discussions are welcome in our github discussions. Release notes and Change Log See also additional deployment/install release notes in wheel_archives

Thursday, June 6 - v0.3.1-WIP Added module 3 to Fast Start example series examples 7-9 on Agents & Function Calls Added reranker Jina model for in-memory semantic similarity RAG - see example Changes merged into main branch - expected next pypi release at end of week

Tuesday, June 4 - v0.3.0 Added support for new Milvus Lite embedded 'no-install' database - see example. Added two new SLIM models to catalog and agent processes - 'q-gen' and 'qa-gen' Updated model class instantiation to provide more extensibility to add new classes in different modules New welcome_to_llmware.sh and welcome_to_llmware_windows.sh fast install scripts Enhanced Model class base with new configurable post_init and register methods Created InferenceHistory to track global state of all inferences completed Multiple improvements and updates to logging at module level Note: starting with v0.3.0, pip install provides two options - a base minimal install pip3 install llmware which will support most use cases, and a larger install pip3 install 'llmware[full]' with other commonly-used libraries.

Wednesday, May 22 - v0.2.15 Improvements in Model class handling of Pytorch and Transformers dependencies (just-in-time loading, if needed) Expanding API endpoint options and inference server functionality - see new client access options and server_launch

Saturday, May 18 - v0.2.14 New OCR image parsing methods with example Adding first part of logging improvements (WIP) in Configs and Models. New embedding model added to catalog - industry-bert-loans. Updates to model import methods and configurations.

Sunday, May 12 - v0.2.13 New GGUF streaming method with basic example and phi3 local chatbot Significant cleanups in ancillary imports and dependencies to reduce install complexity - note: the updated requirements.txt and setup.py files. Defensive code to provide informative warning of any missing dependencies in specialized parts of the code, e.g., OCR, Web Parser. Updates of tests, notice and documentation.

OpenAIConfigs created to support Azure OpenAI.

Sunday, May 5 - v0.2.12 Update Launched "bling-phi-3" and "bling-phi-3-gguf" in ModelCatalog - newest and most accurate BLING/DRAGON model New long document

summarization method using slim-summary-tool example New Office (Powerpoint, Word, Excel) sample files example Added support for Python 3.12 Deprecated faiss and replaced with 'no-install' chromadb in Fast Start examples Refactored Datasets, Graph and Web Services classes Updated Voice parsing with WhisperCPP into Library Monday, April 29 - v0.2.11 Update Updates to gguf libs for Phi-3 and Llama-3 Added Phi-3 example and Llama-3 example and Quantized Versions to Model Catalog Integrated WhisperCPP Model class and prebuilt shared libraries - getting-started-example New voice sample files for testing - example Improved CUDA detection on Windows and safety checks for older Mac OS versions Monday, April 22 - v0.2.10 Update Updates to Agent class to support Natural Language queries of Custom Tables on Postgres example New Agent API endpoint implemented with LLMWare Inference Server and new Agent capabilities example Tuesday, April 16 - v0.2.9 Update New CustomTable class to rapidly create custom DB tables in conjunction with LLM-based workflows. Enhanced methods for converting CSV and JSON/JSONL files into DB tables. See new examples Creating Custom Table example Tuesday, April 9 - v0.2.8 Update Office Parser (Word Docx, Powerpoint PPTX, and Excel XLSX) - multiple improvements - new libs + Python method. Includes: several fixes, improved text chunking controls, header text extraction and configuration options. Generally, new office parser options conform with the new PDF parser options. Please see Office Parsing Configs example Wednesday, April 3 - v0.2.7 Update PDF Parser - multiple improvements - new libs + Python methods. Includes: UTF-8 encoding for European languages. Includes: Better text chunking controls, header text extraction and configuration options. Please see PDF Parsing Configs example for more details. Note: deprecating support for aarch64-linux (will use 0.2.6 parsers). Full support going forward for Linux Ubuntu20+ on x86_64 + with CUDA. Friday, March 22 - v0.2.6 Update New SLIM models: summary, extract, xsum, boolean, tags-3b, and combo sentiment-ner. New logit and sampling analytics. New SLIM examples showing how to use the new models. Thursday, March 14 - v0.2.5 Update Improved support for GGUF on CUDA (Windows and Linux), with new prebuilt binaries and exception handling. Enhanced model configuration options (sampling, temperature, top logit capture). Added full back-level support for Ubuntu 20+ with parsers and GGUF engine. Support for new Anthropic Claude 3 models. New retrieval methods: document_lookup and

aggregate_text. New model: bling-stablelm-3b-tool - fast, accurate 3b quantized question-answering model - one of our new favorites.

Wednesday, February 28 - v0.2.4 Update Major upgrade of GGUF

Generative Model class - support for Stable-LM-3B, CUDA build options, and better control over sampling strategies. Note: new GGUF llama.cpp built libs packaged with build starting in v0.2.4. Improved GPU support for

HF Embedding Models. Friday, February 16 - v0.2.3 Update Added 10+ embedding models to ModelCatalog - nomic, jina, bge, gte, ember and uae-large. Updated OpenAI support >=1.0 and new text-3 embedding models.

SLIM model keys and output_values now accessible in ModelCatalog.

Updating encodings to 'utf-8-sig' to better handle txt/csv files with bom.

Supported Operating Systems: MacOS (Metal and x86), Linux (x86 and aarch64), Windows note on Linux: we test most extensively on Ubuntu 22 and now Ubuntu 20 and recommend where possible if you need another

Linux version, please raise an issue - we will prioritize testing and ensure support. Supported Vector Databases: Milvus, Postgres (PGVector), Neo4j, Redis, LanceDB, ChromaDB, Qdrant, FAISS, Pinecone, Mongo Atlas Vector Search Supported Text Index Databases: MongoDB, Postgres, SQLite

Optional Docker To enable the OCR parsing capabilities, install Tesseract

v5.3.3 and Poppler v23.10.0 native packages. Change Log Latest Updates -

19 Jan 2024 - llmware v0.2.0 Added new database integration options -

Postgres and SQLite Improved status update and parser event logging options for parallelized parsing Significant enhancements to interactions between Embedding + Text collection databases Improved error exception handling in loading dynamic modules Latest Updates - 15 Jan 2024:

llmware v0.1.15 Enhancements to dual pass retrieval queries Expanded configuration objects and options for endpoint resources Latest Updates -

30 Dec 2023: llmware v0.1.14 Added support for Open Chat inference servers (compatible with OpenAI API) Improved capabilities for multiple embedding models and vector DB configurations Added docker-compose install scripts for PGVector and Redis vector databases Added 'bling-tiny-

llama' to model catalog Latest Updates - 22 Dec 2023: llmware v0.1.13

Added 3 new vector databases - Postgres (PG Vector), Redis, and Qdrant

Improved support for integrating sentence transformers directly in the model catalog Improvements in the model catalog attributes Multiple new

Examples in Models & Embeddings, including GGUF, Vector database, and

model catalog 17 Dec 2023: llmware v0.1.12 dragon-deci-7b added to

catalog - RAG-finetuned model on high-performance new 7B model base from Deci New GGUFGenerativeModel class for easy integration of GGUF Models Adding prebuilt llama_cpp / ctransformer shared libraries for Mac M1, Mac x86, Linux x86 and Windows 3 DRAGON models packaged as Q4_K_M GGUF models for CPU laptop use (dragon-mistral-7b, dragon-llama-7b, dragon-yi-6b) 4 leading open source chat models added to default catalog with Q4_K_M 8 Dec 2023: llmware v0.1.11 New fast start examples for high volume Document Ingestion and Embeddings with Milvus. New LLMWare 'Pop up' Inference Server model class and example script. New Invoice Processing example for RAG. Improved Windows stack management to support parsing larger documents. Enhancing debugging log output mode options for PDF and Office parsers. 30 Nov 2023: llmware v0.1.10 Windows added as a supported operating system. Further enhancements to native code for stack management. Minor defect fixes. 24 Nov 2023: llmware v0.1.9 Markdown (.md) files are now parsed and treated as text files. PDF and Office parser stack optimizations which should avoid the need to set ulimit -s. New llmware_models_fast_start.py example that allows discovery and selection of all llmware HuggingFace models. Native dependencies (shared libraries and dependencies) now included in repo to facilitate local development. Updates to the Status class to support PDF and Office document parsing status updates. Minor defect fixes including image block handling in library exports. 17 Nov 2023: llmware v0.1.8 Enhanced generation performance by allowing each model to specify the trailing space parameter. Improved handling for eos_token_id for llama2 and mistral. Improved support for Hugging Face dynamic loading New examples with the new llmware DRAGON models. 14 Nov 2023: llmware v0.1.7 Moved to Python Wheel package format for PyPi distribution to provide seamless installation of native dependencies on all supported platforms. ModelCatalog enhancements: OpenAI update to include newly announced 'turbo' 4 and 3.5 models. Cohere embedding v3 update to include new Cohere embedding models. BLING models as out-of-the-box registered options in the catalog. They can be instantiated like any other model, even without the "hf=True" flag. Ability to register new model names, within existing model classes, with the register method in ModelCatalog. Prompt enhancements: "evidence_metadata" added to prompt_main output dictionaries allowing prompt_main responses to be plug into the evidence and fact-checking steps without modification. API key can now be passed

directly in a prompt.`load_model(model_name, api_key = "[my-api-key]")`

LLMWareInference Server - Initial delivery: New Class for LLMWareModel which is a wrapper on a custom HF-style API-based model.

LLMWareInferenceServer is a new class that can be instantiated on a remote (GPU) server to create a testing API-server that can be integrated into any Prompt workflow. 03 Nov 2023: llmware v0.1.6 Updated packaging to require mongo-c-driver 1.24.4 to temporarily workaround segmentation fault with mongo-c-driver 1.25. Updates in python code needed in anticipation of future Windows support. 27 Oct 2023: llmware v0.1.5 Four new example scripts focused on RAG workflows with small, fine-tuned instruct models that run on a laptop (llmware BLING models). Expanded options for setting temperature inside a prompt class. Improvement in post processing of Hugging Face model generation. Streamlined loading of Hugging Face generative models into prompts. Initial delivery of a central status class: read/write of embedding status with a consistent interface for callers. Enhanced in-memory dictionary search support for multi-key queries. Removed trailing space in human-bot wrapping to improve generation quality in some fine-tuned models. Minor defect fixes, updated test scripts, and version update for Werkzeug to address dependency security alert. 20 Oct 2023: llmware v0.1.4 GPU support for Hugging Face models. Defect fixes and additional test scripts. 13 Oct 2023: llmware v0.1.3 MongoDB Atlas Vector Search support. Support for authentication using a MongoDB connection string. Document summarization methods. Improvements in capturing the model context window automatically and passing changes in the expected output length. Dataset card and description with lookup by name. Processing time added to model inference usage dictionary. Additional test scripts, examples, and defect fixes. 06 Oct 2023: llmware v0.1.1 Added test scripts to the github repository for regression testing. Minor defect fixes and version update of Pillow to address dependency security alert. 02 Oct 2023: llmware v0.1.0 Initial release of llmware to open source!!